The Journal of Open Source Software

POMDPPlanners: Open-Source Package for POMDP Planning

Yaacov Pariente[1], **Vadim Indelman**[2,3]

1 Faculty of Mathematics, Technion - Israel Institute of Technology **2** Stephen B. Klein Faculty of Aerospace Engineering, Technion - Israel Institute of Technology **3** Faculty of Data and Decision Sciences, Technion - Israel Institute of Technology

DOI: 10.xxxxxx/draft

Software

- Review
- Repository
- Archive

Editor: Open Journals

Reviewers:

- @openjournals

Submitted: 01 January 1970
Published: unpublished

License
Authors of papers retain copyright and release the work under a Creative Commons Attribution 4.0 International License (CC BY 4.0).

Summary

POMDPPlanners is an open-source Python package for empirical evaluation of Partially Observable Markov Decision Process (POMDP) planning algorithms. The package integrates state-of-the-art planning algorithms, a suite of benchmark environments with safety-critical variants, automated hyperparameter optimization, persistent caching with failure recovery, and configurable parallel simulation—reducing the overhead of extensive simulation studies. POMDPPlanners is designed to enable scalable, reproducible research on decision-making under uncertainty, with particular emphasis on risk-sensitive settings, which standard toolkits do not support.

Statement of need

POMDPs model sequential decision-making under uncertainty, with applications spanning robotics, autonomous navigation, and medical decision-making. Conducting robust, reproducible simulation studies, however, remains practically challenging: researchers must manually integrate planners, belief representations, and environments from disparate sources, implement hyperparameter search routines, and manage interrupted campaigns—all without standardized tooling in Python. Existing frameworks include C++ toolkits (Bargiacchi et al., 2020; Oliehoek et al., 2017) and the Julia-based POMDPs.jl (Egorov et al., 2017), while Python alternatives (Emami et al., 2015; Zheng & Tellex, 2020) implement only basic solvers and do not support continuous state, observation, and action spaces. POMDPPlanners addresses these gaps with a unified, extensible framework for POMDP planning research. The main contributions of the package are:

1. **A unified workflow for reproducible simulation studies:** Automated hyperparameter tuning, parallel evaluation on local machines and remote clusters, persistent caching, and recovery from interrupted runs reduce experimental setup work. A local web interface supports comparing results, replaying episodes, and customizing and exporting figures for academic papers.
2. **Benchmarks for safety and scalability:** Configurable hazards and safety metrics extend established POMDP environments for evaluating risk-averse planners. New environments with adjustable state, observation, and action space sizes allow researchers to assess performance as problem size increases.
3. **Integration with realistic simulators:** Common environment interfaces for CARLA and Isaac Lab support autonomous navigation and robot manipulation studies while reducing simulator-specific integration work.
4. **A shared interface for planning algorithms:** Implementations of standard and risk-averse planners use common interfaces for environments, beliefs, and hyperparameter tuning, making it easier to compare algorithms under consistent experimental settings.

DRAFT

State of the field

Solving POMDP planning problems relies on computing large search trees that grow exponentially with the horizon, and are in general intractable (Papadimitriou & Tsitsiklis, 1987). To address this complexity, online POMDP planners approximate search trees by simulating future trajectories during execution (Silver & Veness, 2010; Sunberg & Kochenderfer, 2018), making computational efficiency a priority. These simulation intensive algorithms pushed the field of POMDP planning to use efficient programming languages like C++ (Bargiacchi et al., 2020; Oliehoek et al., 2017) and Julia (Egorov et al., 2017), where most of the new literature is in Julia (Moss, Corso, et al., 2024; Moss, Jamgochian, et al., 2024). These computationally efficient programming languages prioritize efficiency over usage simplicity, thereby forcing researchers to focus on coding instead of research, thereby extending the delivery time of simulation studies for academic papers. In POMDPPlanners we resolve the efficiency problem by implementing vectorized belief updates for each environment in Python, and move environment-specific code to C++ while keeping core interfaces in Python. Recent work such as VOPP demonstrates fully vectorized online POMDP planning on GPUs using Python and PyTorch, further motivating Python-based planner implementations (Hoerger et al., 2025).

The two existing POMDP planning python packages (Emami et al., 2015; Zheng & Tellex, 2020) are outdated, and implement only a small amount of simple planners that cannot solve POMDPs with continuous spaces, which is required to solve real-world problems. Table 1 summarizes the key differences between our work and previous software tools.

Table 1: Comparison of POMDP planning frameworks. ✓ = supported; – = not supported. All frameworks support discrete spaces; Cont. = continuous spaces; Hyp. opt. = automated hyperparameter optimization; Safety Envs. = safety-critical environments with violation metrics; Modern Planners = includes planners beyond POMCP appearing in post-2018 POMDP literature (e.g., POMCPOW, PFT-DPW, BetaZero). †POMDPs.jl provides a solver interface; planners are separate community packages.

Package	Language	Cont.	Modern Planners	Hyp. opt.	Safety Envs.	Parallel	Caching
AI-Toolbox	C++	–	–	–	–	–	–
MADP	C++	–	–	–	–	–	–
POMDPs.jl	Julia	✓	✓†	–	–	✓	–
pomdpy	Python	–	–	–	–	–	–
pomdp_py	Python	–	–	–	–	–	–
POMDPPlanners	**Python**	✓	✓	✓	✓	✓	✓

Software Design

The design of the package revolves around the problem it solves: comparing POMDP planning algorithms through large-scale simulations. This problem can be divided into three parts: comparable planner implementations, reliable execution of large simulation studies, and detailed analysis of results.

Comparing planners can be highly affected by data structures choices rather than algorithmic differences. To solve this we implemented efficient tree data structures that performed well during our development. All of our tree-based planners always use the most efficient tree data structure that we know of, thereby improving all of the implemented planners in our package and removing data-structure dependencies when comparing planning algorithms.

Evaluating these planners across environments and parameter settings then requires reliable execution of large simulation studies. The package combines automated parameter tuning using Optuna (Akiba et al., 2019) with parallel execution on local machines and remote clusters

DRAFT

using the Dask Python package. Persistent caching stores the results of completed episodes so that interrupted studies can resume without repeating completed work. Each episode runs as a standalone task wrapped with command pattern, and all of the tasks are dispatched using task-manager. The user runs simulations though the simulations API, and is agnostic to the implementation as specified in Figure 1.

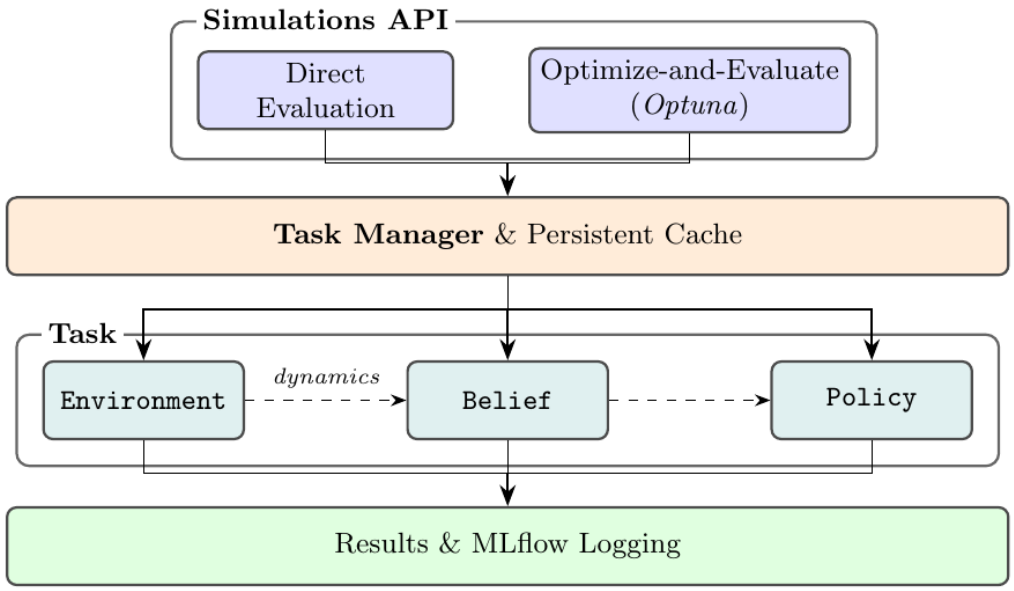


Figure 1: Architecture of POMDPPlanners. Evaluation workflows share a task manager and persistent cache; results are logged through MLflow. Dashed arrows show data flow between the core abstractions.

Interpreting the results of these studies requires understanding what happens within each planner run. For example, a low mean return may reflect insufficient exploration caused by a poorly tuned exploration constant. The package therefore records a range of planner and environment metrics for each episode. These metrics are integrated easily through the environment and planner interfaces. Examples include action coverage, task completion, and safety violations, which help researchers diagnose planning behavior and assess success rates and risk. Cached trajectory visualizations complement these metrics: an agent that avoids every obstacle may simply have remained stationary.

AI usage disclosure

AI tools were used in the developlemt of the package and for compact rephrasing of this paper. All of the results generated by these tools were reviewed.

Acknowledgements

This work was supported by the Israel Ministry of Innovation, Science and Technology.

References

Akiba, T., Sano, S., Yanase, T., Ohta, T., & Koyama, M. (2019). Optuna: A next-generation hyperparameter optimization framework. *The 25th ACM SIGKDD International Conference on Knowledge Discovery & Data Mining*, 2623–2631.

DRAFT

Bargiacchi, E., Roijers, D. M., & Nowé, A. (2020). AI-toolbox: A c++ library for reinforcement learning and planning (with python bindings). *Journal of Machine Learning Research*, *21*(102), 1–12. http://jmlr.org/papers/v21/18-402.html

Egorov, M., Sunberg, Z. N., Balaban, E., Wheeler, T. A., Gupta, J. K., & Kochenderfer, M. J. (2017). POMDPs.jl: A framework for sequential decision making under uncertainty. *Journal of Machine Learning Research*, *18*(26), 1–5. http://jmlr.org/papers/v18/16-300.html

Emami, P., Hamlet, A. J., & Crane, C. (2015). *POMDPy: An extensible framework for implementing POMDPs in python*.

Hoerger, M., Sudrajat, M., & Kurniawati, H. (2025). Vectorized online POMDP planning. *arXiv Preprint arXiv:2510.27191*. https://doi.org/10.48550/arXiv.2510.27191

Moss, R. J., Corso, A., Caers, J., & Kochenderfer, M. J. (2024). BetaZero: Belief-State Planning for Long-Horizon POMDPs using Learned Approximations. *Reinforcement Learning Conference (RLC)*.

Moss, R. J., Jamgochian, A., Fischer, J., Corso, A., & Kochenderfer, M. J. (2024). ConstrainedZero: Chance-Constrained POMDP Planning Using Learned Probabilistic Failure Surrogates and Adaptive Safety Constraints. *International Joint Conference on Artificial Intelligence (IJCAI)*.

Oliehoek, F. A., Spaan, M. T. J., Terwijn, B., Robbel, P., & Messias, J. V. (2017). The MADP toolbox: An open source library for planning and learning in (multi-)agent systems. *Journal of Machine Learning Research*, *18*(89), 1–5. http://jmlr.org/papers/v18/17-156.html

Papadimitriou, C. H., & Tsitsiklis, J. N. (1987). The complexity of Markov decision processes. *Mathematics of Operations Research*, *12*(3), 441–450. https://doi.org/10.1287/moor.12.3.441

Silver, D., & Veness, J. (2010). Monte-carlo planning in large POMDPs. *Advances in Neural Information Processing Systems*, *23*.

Sunberg, Z., & Kochenderfer, M. (2018). Online algorithms for POMDPs with continuous state, action, and observation spaces. *Proceedings of the International Conference on Automated Planning and Scheduling*, *28*, 259–263.

Zheng, K., & Tellex, S. (2020). Pomdp_py: A framework to build and solve POMDP problems. *ICAPS 2020 Workshop on Planning and Robotics (PlanRob)*. https://icaps20subpages.icaps-conference.org/wp-content/uploads/2020/10/14-PlanRob_2020_paper_3.pdf

DRAFT